

In [12]:

```
from abc import ABC, abstractmethod # ABC --> Abstract Base classes

class Bank (ABC):
    def getroi(self):
        pass

class SBI (Bank):
    def getroi(self):
        print("SBI Bank")
        return 4.5

class ICICI (Bank):
    def getroi(self):
        print("ICICI Bank")
        return 6.0

class HDFC (Bank):
    def getroi(self):
        print("HDFC Bank")
        return 6.5

s1 = SBI()
i1 = ICICI()
h1 = HDFC()

print(s1.getroi())
print(i1.getroi())
print(h1.getroi())
```

SBI Bank
4.5
ICICI Bank
6.0
HDFC Bank
6.5

In [13]:

```
from abc import ABC

class Figure (ABC):
    def show(self):
        pass
    def area(self, a , b):
        pass

class Rectangle (Figure):
    def area(self, a, b):
        return a*b
    def show(self):
        print("Rectangle")

class Triangle (Figure):
    def area(self, a, b):
        return 0.5*a*b
    def show(self):
        print("Triangle")

r1 = Rectangle()
t1 = Triangle()

a = 10
b = 20

r1.show()
print("Area of rectangle = " , r1.area(a,b))

t1.show()
print("Area of triangle = " , t1.area(a,b))
```

```
Rectangle
Area of rectangle = 200
Triangle
Area of triangle = 100.0
```

In [18]:

```
from abc import ABC

class Figure (ABC):
    def show(self):
        pass
    def area(self, a , b):
        print("Area of Figure")
        return a*b

class Rectangle (Figure):
    def show(self):
        print("Rectangle")

class Triangle (Figure):
    def area(self, a, b):
        print("Area of Triangle")
        return 0.5*a*b
    def show(self):
        print("Triangle")

r1 = Rectangle()
t1 = Triangle()

a = 10
b = 20

r1.show()
print(r1.area(a,b))

t1.show()
print(t1.area(a,b))
```

```
Rectangle
Area of Figure
200
Triangle
Area of Triangle
100.0
```

In [25]:

```
#Stack and Queue

def declare_stack():
    stack = []
    return stack

def isempty():
    return len(stack) == 0

def push(stack,item):
    stack.append(item)
    print("Item pushed to stack = ", item)

def pop(stack):
    if isempty():
        print("Stack is empty")
    else:
        stack.pop()

stack = declare_stack()
push(stack, "Java")
push(stack, "C++")
push(stack, "Python")

print(stack)

pop(stack)
print(stack)

pop(stack)
print(stack)

pop(stack)
print(stack)

pop(stack)
print(stack)

push(stack, "Java")
print(stack)
```

```
Item pushed to stack = Java
Item pushed to stack = C++
Item pushed to stack = Python
['Java', 'C++', 'Python']
['Java', 'C++']
['Java']
[]
Stack is empty
[]
Item pushed to stack = Java
['Java']
```

In [36]:

```

class Queue:
    def __init__(self):
        self.queue = []

    #add the element
    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        if len(self.queue) < 1:
            return -1
        return self.queue.pop(0)
    def display(self):
        print("Queue = " , self.queue)

q1 = Queue()
q1.display()

for var in range(1,5):
    q1.enqueue(var)
q1.display()

print("Item removed = " , q1.dequeue())
q1.display()

print("Item removed = " , q1.dequeue())
q1.display()

for var in range(10,20):
    if var %2 ==0:
        q1.enqueue(var)

q1.display()

print("Item removed = " , q1.dequeue())
q1.display()

```

```

Queue = []
Queue = [1, 2, 3, 4]
Item removed = 1
Queue = [2, 3, 4]
Item removed = 2
Queue = [3, 4]
Queue = [3, 4, 10, 12, 14, 16, 18]
Item removed = 3
Queue = [4, 10, 12, 14, 16, 18]

```

In [42]:

```
# Sorting
#Bubble Sort
# 10 20
# 30 20
# 30 10
# 20 10

def bubblesort(array):
    for i in range(len(array)):
        for j in range(0, len(array)-i-1):
            if array[j] > array[j+1]:
                array[j] = array[j] + array[j+1]
                array[j+1] = array[j] - array[j+1]
                array[j] = array[j] - array[j+1]
        print(array)

array = [45,56,32,23,37,10,8]
bubblesort(array)

print('Sorted array = ', array)
```

```
[45, 32, 23, 37, 10, 8, 56]
[32, 23, 37, 10, 8, 45, 56]
[23, 32, 10, 8, 37, 45, 56]
[23, 10, 8, 32, 37, 45, 56]
[10, 8, 23, 32, 37, 45, 56]
[8, 10, 23, 32, 37, 45, 56]
[8, 10, 23, 32, 37, 45, 56]
Sorted array = [8, 10, 23, 32, 37, 45, 56]
```

In [45]:

```
#insertion Sort
def insertionsort(array):
    for step in range (1, len(array)):
        key = array[step]
        j = step-1

        while j >=0 and key < array[j]:
            array[j+1] = array[j]
            j = j-1
        array[j+1] = key
        print(array)

array = [45,56,32,23,37,10,8]
insertionsort(array)

print('Sorted array = ', array)
```

```
[45, 56, 32, 23, 37, 10, 8]
[32, 45, 56, 23, 37, 10, 8]
[23, 32, 45, 56, 37, 10, 8]
[23, 32, 37, 45, 56, 10, 8]
[10, 23, 32, 37, 45, 56, 8]
[8, 10, 23, 32, 37, 45, 56]
Sorted array = [8, 10, 23, 32, 37, 45, 56]
```

In []:

```
#Selection sort
#Merge sort
#Quick Sort
#Bucket sort
#Heap Sort

#Serching
#Linear Search
# Binary Search (List must be already sorted)
```

In [63]:

```
def linearsearch(array, item):
    status = False
    for var in array:
        if var == item:
            status = True
    if status:
        print('Item Found')
    else:
        print("Item Not Found")

array = [45,56,32,23,37,10,8]
linearsearch(array, int(input("Enter the item to search in array")))
```

Enter the item to search in array56
Item Found

In [65]:

```
def linearsearch(array, item):
    status = False
    index = -1
    for i in range(0,len(array)):
        if array[i] == item:
            status = True
            index = i
    if status:
        print('Item Found at index', index)
    else:
        print("Item Not Found")

array = [45,56,32,23,37,10,8]
linearsearch(array, int(input("Enter the item to search in array")))
```

Enter the item to search in array456
Item Not Found

In []: